

Class 05: React Events, User Interaction, Forms

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

React Events



- React รองรับการกระทำ actions เพื่อตอบสนองต่อ events จาก user หรือสิ่งแวดล้อม เช่นเดียวกับ HTML DOM events
  - React ทำงานกับ Event ชุดเดียวกับ HTML Events เช่น `click`, `change`, `load`, `unload`, `mouseover`, `mouseout` เป็นต้น
- React Event เขียนด้วย camel case ตัวอย่างเช่น:
  - HTML Event Attribute `onclick` → React Event `onClick`
  - HTML Event Attribute `onchange` → React Event `onChange`
  - HTML Event Attribute `onload` → React Event `onLoad`
  - HTML Event Attribute `onmouseover` → React Event `onMouseOver`
- React Event Handler เขียนอยู่ภายใน { } ตัวอย่างเช่น
  - HTML Event

```
<div onclick="alert('Hello World!')">Click Me</div>
```

  - React Event

```
<div onClick={()=>alert("Hello World!")}> Click Me</div>
```

React Events



- **React Event Handler**
  เรียกใช้งาน function ได้

```
export default function Home() {
  function greet() {
    alert("Hello World!")
  }

  return (
    <div onClick={()=>greet()}> Click Me</div>
  )
}
```

- Passing Argument

```
export default function Home() {
  function greet(message:string) {
    alert(message)
  }

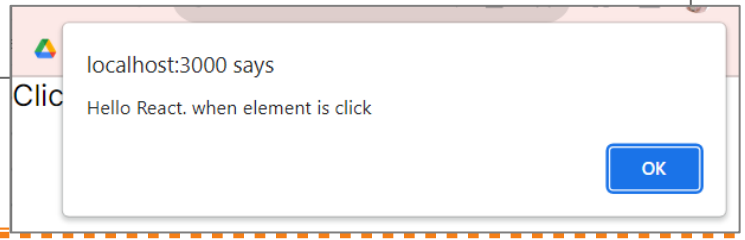
  return (
    <div onClick={()=>greet("Hello React.")}> Click Me</div>
  )
}
```

React Events

- **Event Object**
  - Event handlers รับ Event ที่ trigger ตัว Handler ผ่าน Event Object ซึ่งบรรจุข้อมูลต่าง ๆ เกี่ยวกับ Event นั้น ๆ

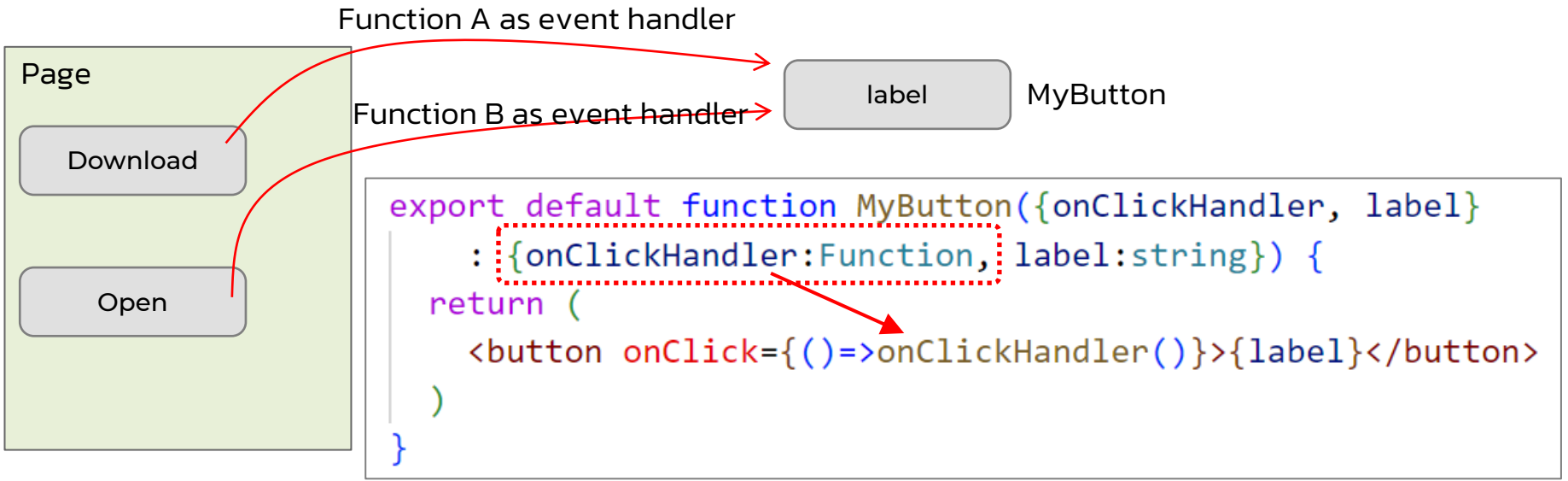
```
import React from "react"
export default function Home() {
  function greet(message:string, e:React.SyntheticEvent) {
    alert(message + " when element is " + e.type)
  }

  return (
    <div onClick={(e)=>greet("Hello React.", e)}> Click Me</div>
  )
}
```



React Events

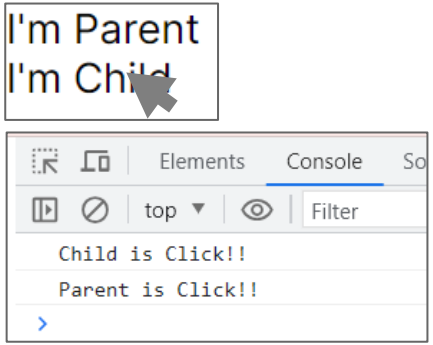
- **Pass event handlers เป็น Component props**
  - ในบางครั้ง เราต้องการให้ Component จัดการ Event แตกต่างกันไป ตามที่ Parent Component กำหนด
    - Button Component เมื่อนำไปใช้ในหน้าหนึ่ง อาจใช้ในการส่งข้อมูล แต่เมื่อเอาไปใช้อีกหน้า อาจใช้ในการเปิด Dialog Box



React Events

- **Event Propagation**
  - เมื่อ Child Component ได้รับ Event จะ propagate (bubble) Event นั้น ขึ้นไปยัง parent component ด้วย

```
export default function Home() {
  return (
    <div onClick={()=>console.log("Parent is Click!!")}>
      I'm Parent
      <div onClick={()=>console.log("Child is Click!!")}>I'm Child</div>
    </div>
  )
}
```

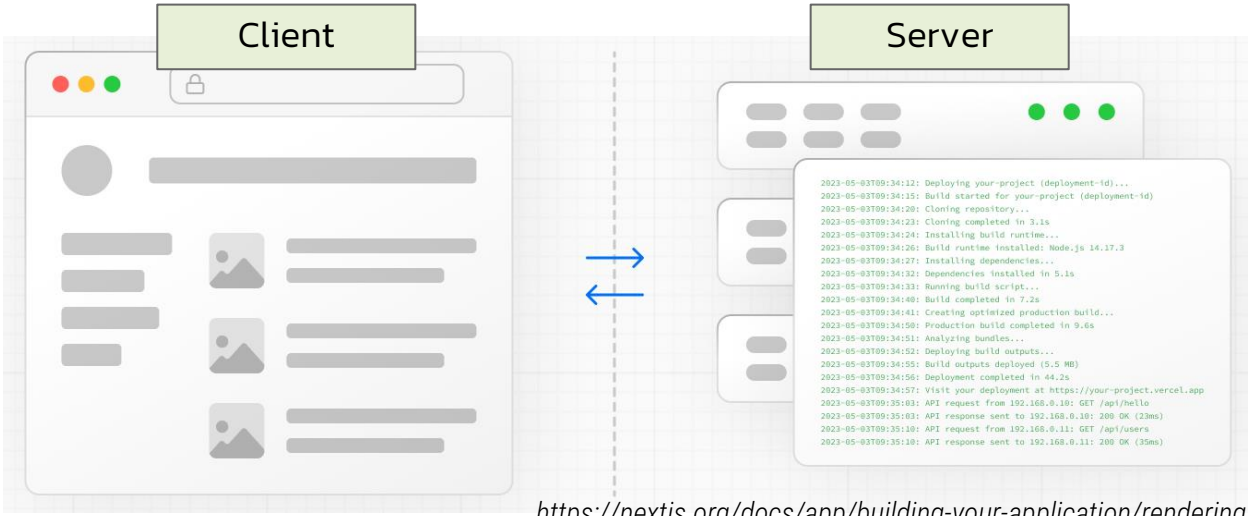


  - หยุดได้ด้วยการเรียก `stopPropagation()` บน event object

```
export default function Home() {
  return (
    <div onClick={()=>console.log("Parent is Click!!")}>
      I'm Parent
      <div onClick={(e)=>{console.log("Child is Click!!")
      e.stopPropagation()}}>I'm Child</div>
    </div>
  )
}
```

Next.js Components Interaction

- Next.js รองรับการ render component บนทั้ง 2 environment : Client และ Server

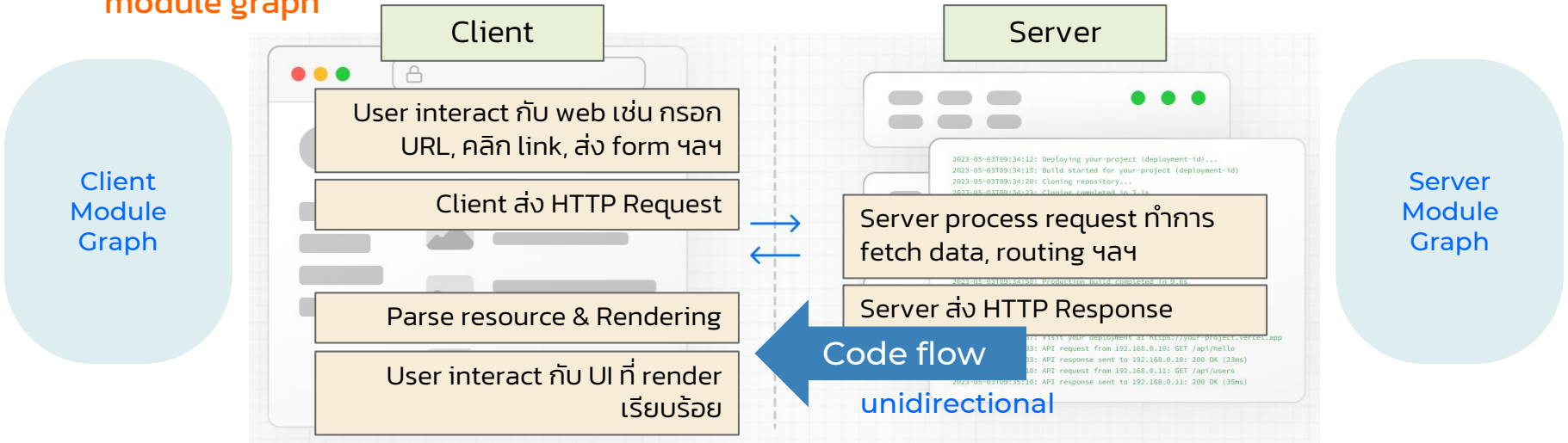


https://nextjs.org/docs/app/building-your-application/rendering

- ใช้ภาษาเดียวกัน เช่น JavaScript, TypeScript บนทั้ง Client และ Server
- อย่างไรก็ตาม แต่ละ environment มีความสามารถ เงื่อนไข และข้อจำกัดต่างกันไป
  - การ fetch data, การบริหารจัดการ user state

Next.js Components Interaction

- **Network Boundary สำหรับ Hybrid Applications**
  - การทำงานกับ Next.js Components อาศัยการออกแบบเส้นแบ่งระหว่าง Client environment และ Server environment (Network Boundary)
  - ใน application มีการเชื่อมโยงและพึ่งพากัน (dependencies) ของไฟล์ต่าง ๆ ซึ่งนำเสนอได้ในรูปแบบของ module graph



Next.js Components Interaction



- Network Boundary สำหรับ Hybrid Applications
  - React กำหนด Network Boundary ด้วยการระบุผ่าน convention `"use client"`
  - React รองรับการทำ computational job บนฝั่ง server ขณะอยู่บน client ด้วยการระบุผ่าน convention `"use server"`
- Server Component
  - React Server Components รองรับการเขียน UI ที่ถูก render และอาจถูก cache อยู่บน server
  - Next.js แยกการทำ rendering ด้วย route segment เพื่อรองรับการทำ partial rendering และการทำ streaming
  - Next.js ใช้ server component โดย default
  - Server Rendering ให้ประโยชน์ต่าง ๆ สำหรับ Next.js application
    - Data Fetching ที่มีประสิทธิภาพขึ้น เพราะการใช้ server component เป็นการ fetch data ตั้งแต่บน server ซึ่งใกล้กับ data source มากกว่า
    - Security ที่ดีกว่าสำหรับ component ที่มี sensitive data เช่น token, api key รวมถึง sensitive logic
    - Caching component บน server สำหรับการให้บริการ request อื่นในอนาคต เพิ่มประสิทธิภาพการให้บริการ

Next.js Components Interaction



- **Server Component**
  - **Bundle Size** ให้ขนาดของ client JavaScript bundle เล็กลง โดยเฉพาะเมื่อใช้ dependencies ขนาดใหญ่
  - **Initial Page Load** ใช้เวลาโหลดครั้งแรกลดลง เพราะมีการ generate HTML ตั้งแต่ server หลีกเลี่ยงการ parse และ execute JavaScript บน client
  - **Search Engine Optimization (SEO) & Social Network Shareability** ที่ดีกว่า หน้า HTML ที่ render แล้วตั้งแต่ server ทำให้ search engine bots สร้าง index สำหรับหน้าเพจ และทำให้ social network bots สร้าง social card previews ต่าง ๆ สำหรับหน้าเพจ
  - **Streaming** component บน server สามารถถูกแบ่งเป็นชิ้นเล็ก ๆ แล้วทยอย stream ไปยัง client เมื่อ renderพร้อมแล้ว ทำให้ user บน client เห็นหน้าเพจเร็ว ทีละส่วน โดยไม่ต้องรอให้ครบทั้งหน้า
- **Server-Side Rendering**
  - Next.js ใช้ React APIs ประสานงานการ render โดยแบ่งงาน render ออกเป็นชิ้น ๆ โดยอาศัย 2 ขั้นตอน
    - React ทำการ render Server Components ไปเป็น data format พิเศษ เรียกว่า **React Server Component Payload (RSC Payload)**
    - Next.js ใช้ RSC Payload และ Client Component JavaScript instruction เพื่อ render HTML บน server

Next.js Components Interaction



- **Server-Side Rendering**
  - บน Client มีการทำงานต่อไปนี้
    - สำหรับ initial page load มีการใช้ HTML เพื่อแสดง non-interactive preview สำหรับ route อย่างรวดเร็ว
    - React Server Components Payload ประสาน Client Component tree และ Server Component tree และ update DOM
    - ใช้ JavaScript instructions เพื่อเติม event listener ลงไปใน Client Components (เรียกว่า hydration) และทำให้ application interactive
- **Server Rendering Strategies**

  Next.js รองรับกลยุทธ์การทำ Server Rendering 3 รูปแบบ
  - Static Rendering เป็นรูปแบบ default โดย route ได้รับการ render ตั้งแต่ build time หรือหากมีการทำ data revalidation จาก data fetching ก็จะทำการ render ใน background จากนั้นจะมีการ cache ผลลัพธ์ และอาจมีการ push ผลลัพธ์ไปยัง Content Delivery Network (CDN)
    - สนับสนุนการ share ผลลัพธ์ของการ render ระหว่าง user ต่าง ๆ และ server request ต่าง ๆ
    - ดีสำหรับ route ที่มี data ที่ไม่ต้อง personalized สำหรับ user และทราบตั้งแต่ build time เช่น หน้าสินค้า

Next.js Components Interaction

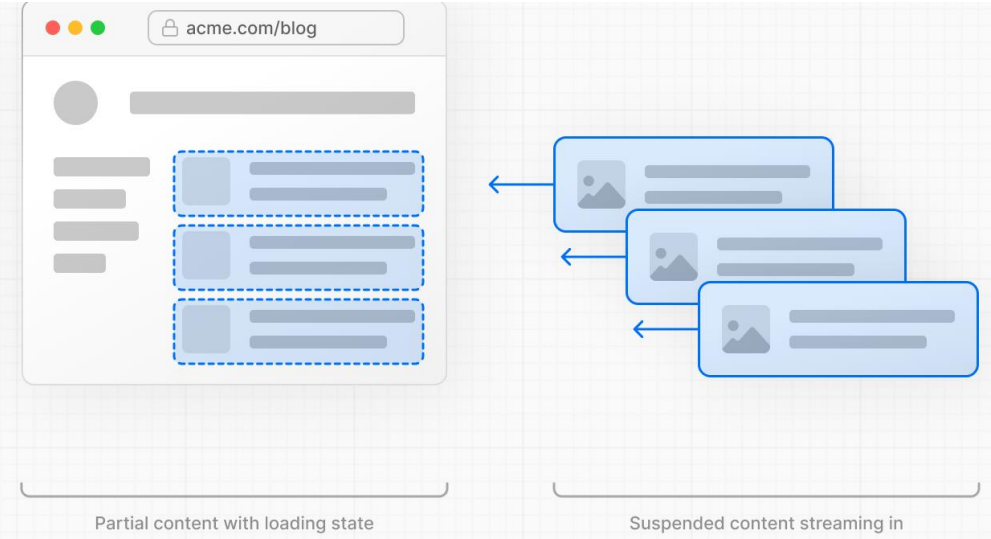


- Server Rendering Strategies
  - Dynamic Rendering Route ได้รับการ render สำหรับแต่ละ user ช่วง request time
    - ดีสำหรับ route ที่มี data ที่ต้อง personalized สำหรับ user และทราบข้อมูลตอน request time เช่น ต้องอาศัยข้อมูลจาก cookies หรือจาก URL search params
    - Next.js รองรับการ switch ไป Dynamic Rendering ถ้าระหว่างการ render พบว่ามี dynamic function หรือ uncached data request โดย switch ไป dynamic rendering สำหรับทั้ง route
    - Developer ไม่ต้องเลือกระหว่าง static rendering กับ dynamic rendering เพราะ Next.js จะปรับกลยุทธ์ให้อัตโนมัติ ตาม features และการใช้งาน APIs

  - Streaming มีการ render route บน server ตอน request time โดยแบ่งงาน rendering เป็นชิ้นเล็ก ๆ แล้วทยอย stream ไปยัง client เมื่อ render พร้อม
    - ทำให้ user บน client เห็นหน้าเพจเร็ว ทีละส่วน โดยไม่ต้องรอให้ครบทั้งหน้า

Next.js Components Interaction

- Server Rendering Strategies
  - Streaming
    - Streaming เป็นประโยชน์สำหรับ UI ที่มี priority ต่ำกว่า UI หลัก หรือ UI ที่อาศัยข้อมูลที่ต้องอาศัยเวลาในการดึงข้อมูล และอาจ block การ render route ทั้ง route



https://nextjs.org/docs/app/building-your-application/rendering/server-components

Next.js Components Interaction



- **Client Component**
  - Client Components รองรับการสร้าง interactive UI ที่รับการ render บน client ตอน request time
  - ใน Next.js นักพัฒนาต้องระบุให้ชัดเจนว่า component ใดที่ React ควร render บน client
  - Client Rendering ให้ประโยชน์ต่าง ๆ สำหรับ Next.js application
    - Interactivity Client Component สนับสนุนการใช้งานกลไก เช่น state, effects รวมถึง event listeners ที่ให้ feedback กลับไปยัง user และ update UI
    - Browser APIs: Client Component สนับสนุนการใช้งาน Browser APIs เช่น local storage, geolocation ซึ่งเป็นการทำงานที่เจาะจงกับแต่ละ user

- **การใช้งาน Client Component**
  - ระบุว่าเป็น Client Component ผ่าน directive `"use client"` ที่ด้านบนสุดของไฟล์ที่เป็น component
    - Component นั้น รวมถึง modules ที่ import เข้ามาใน component ดังกล่าว และ child component ทั้งหมดของ component ดังกล่าว จะกลายเป็นส่วนหนึ่งของ client bundle และถูก render บน client

Next.js Components Interaction



- การใช้งาน Client Component
  - นักพัฒนาสามารถระบุ `"use client"` entry point หลายจุดใน React Component tree ได้
    - เป็นการแบ่ง application ออกเป็น client bundle หลายอัน (หรือมีหลาย branch ใน component tree)
- Client Component Rendering
  - รูปแบบการ Render Client Component ใน Next.js ขึ้นกับลักษณะของ request ว่าเป็นส่วนหนึ่งของ full page load (แรกเข้า application หรือมีการทำ page reload จาก browser refresh) หรือ request เป็นส่วนหนึ่งของ navigation
  - Full Page Load เพื่อเพิ่มประสิทธิภาพของการโหลดหน้าเพจ React APIs จะ render หน้า static HTML preview ตั้งแต่บน server สำหรับทั้ง Client Component และ Server Component

    บน Server
    - React ทำการ render Server Components ไปเป็น data format พิเศษ เรียกว่า React Server Component Payload (RSC Payload) ซึ่งรวม reference ไปยัง Client Component ด้วย
    - Next.js ใช้ RSC Payload และ Client Component JavaScript instruction เพื่อ render HTML สำหรับ route บน server

Next.js Components Interaction



- Client Component Rendering
  - Full Page Load

    บน Client
    - สำหรับ initial page load มีการใช้ HTML เพื่อแสดง non-interactive preview สำหรับ route อย่างรวดเร็ว
    - React Server Components Payload ประสาน Client Component tree และ Server Component tree และ update DOM
    - ใช้ JavaScript instructions เพื่อเติม event listener ลงไปใน Client Components (เรียกว่า hydration) และทำให้ application interactive
  - Navigation ต่อ ๆ มา จะเป็นการ Render Client Component ทั้งหมดบน client โดยไม่มี HTML ที่ render จาก server
    - Client Component JavaScript bundle จะถูก download และ parse
    - เมื่อ bundle พร้อมใช้งาน React จะใช้ RSC Payload เพื่อประสาน Client Component Tree และ Server Component tree และ update DOM

Next.js Components Interaction



- **Client Component Rendering**
  - แม้จะเป็น Client Component ที่มีการใช้ directive `"use client"` ก็ตาม อาจมีการกลับไปอาศัยการทำงานบน server environment
    - เพื่อลดขนาด client bundle size
    - เพื่อ fetch data บน server
    - เพื่อใช้ API บน server
  - อาจเก็บ code ไว้บน server แม้ว่าจะถูกใส่อยู่ใน Client Component ด้วยการ interleave ระหว่าง Client Component และ Server Component และ Server Action

- **Interleaving Server Components & Client Components**
  - มอง UI เป็น Tree ของ component โดยมี Root Layout เป็น Server Component
  - บาง Sub-Tree อาจเป็น Client Component

Next.js Components Interaction



- **Interleaving Server Components & Client Components**
  - ใน Sub-Tree ที่เป็น Client Component ยังสามารถมี Server Component ซ้อนด้านใน หรือเรียก Server Action ได้
    - การติดต่อ Server จาก Client Component ถือเป็น HTTP Request ใหม่
    - ด้วย HTTP request ใหม่ Server Component ทั้งหมด จะถูก render ก่อน รวมถึง Server Component ที่ซ้อนอยู่ภายใน Client Component ด้วย ผลลัพธ์ของการ render (RSC Payload) จะบรรจุ reference ไปยังตำแหน่งของ Client Component เมื่ออยู่บน client แล้ว React ถึงใช้ RSC Payload เพื่อประสาน Server Component และ Client Component เข้าเป็น tree เดียวกัน
  - **Pattern ที่ไม่สนับสนุน**
    - ไม่รองรับการ import Server Component เข้าไปยัง Client Component module
  - **Pattern ที่สนับสนุน**
    - Server Component import Client Component และ import Server Component ได้
    - ส่ง Server Component เป็น props ไปยัง Client Component

Next.js Components Interaction

- Interleaving Server Components & Client Components

```
'use client'
import ServerComponent from './Server-Component'

export default function ClientComponent() {
  return (
     …
  )
}
```



Server Component

```
'use client'

export default function ClientComponent({children,} : { children: React.ReactNode}) {
  return (
     …
  )
}
```



Next.js Components Interaction

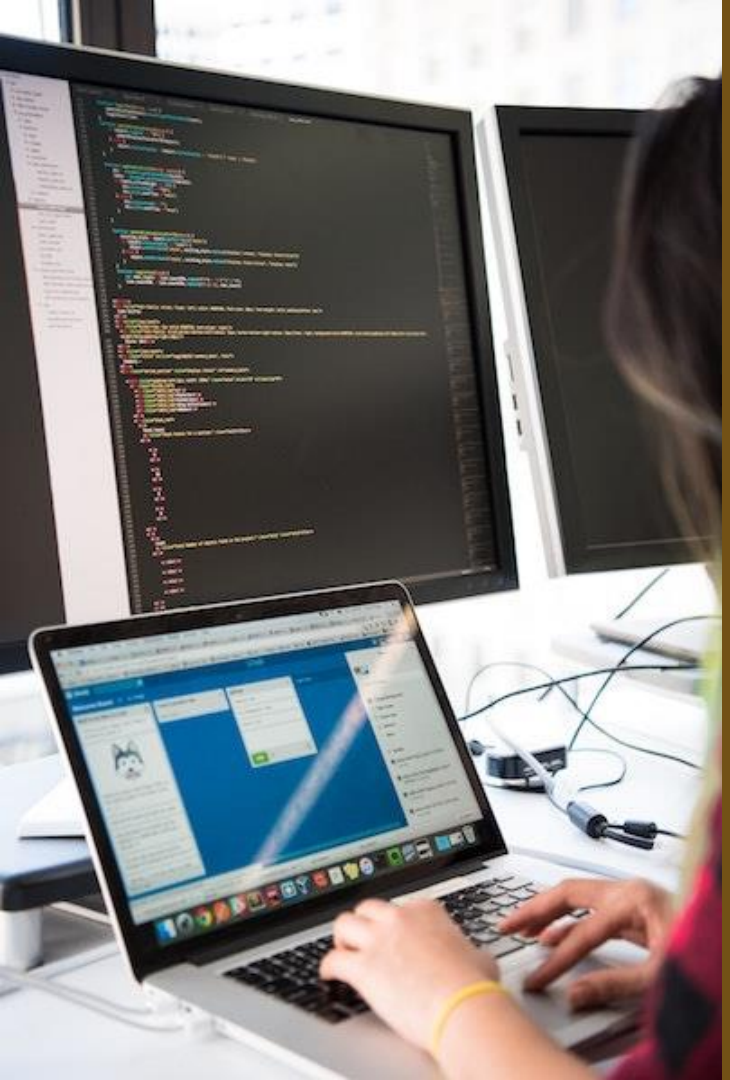


- Interleaving Server Components & Client Components

```
import ServerComponentY from './Server-ComponentY’
import ClientComponent from ‘./Client-Component'

export default function ServerComponentX() {
  return (
     <ClientComponent>
       <ServerComponentY />
    </ClientComponent>
  )
}
```





Workshop 5.1 :
User Interaction & React Events

Interacting with Product Card



- หน้า components/ProductCard.tsx ทดลองทำงานกับ Event บน component นี้
  - ปรับให้ ProductCard.tsx เป็น Client Component โดยการระบุ directive “use client”
  - Handle Event onClick โดยการ alert ข้อความที่เป็นชื่อรุ่นรถ

```
'use client'
import Image from 'next/image';

export default function ProductCard ( { carName, imgSrc } : { carName:string,
imgSrc:string }) {
    return (
        <div className='w-1/5 h-[300px] rounded-lg shadow-lg'
        onClick={()=>alert("You Select " + carName)}>
            <div className='w-full h-[70%] relative rounded-t-lg'>
            <Image src={imgSrc}
                alt='Product Picture'
                fill={true}
                className='object-cover rounded-t-lg'/>
            </div>
```

Interacting with Product Card



- ปรับให้ Handle Event onClick โดยการ เรียกฟังก์ชั่นแทน

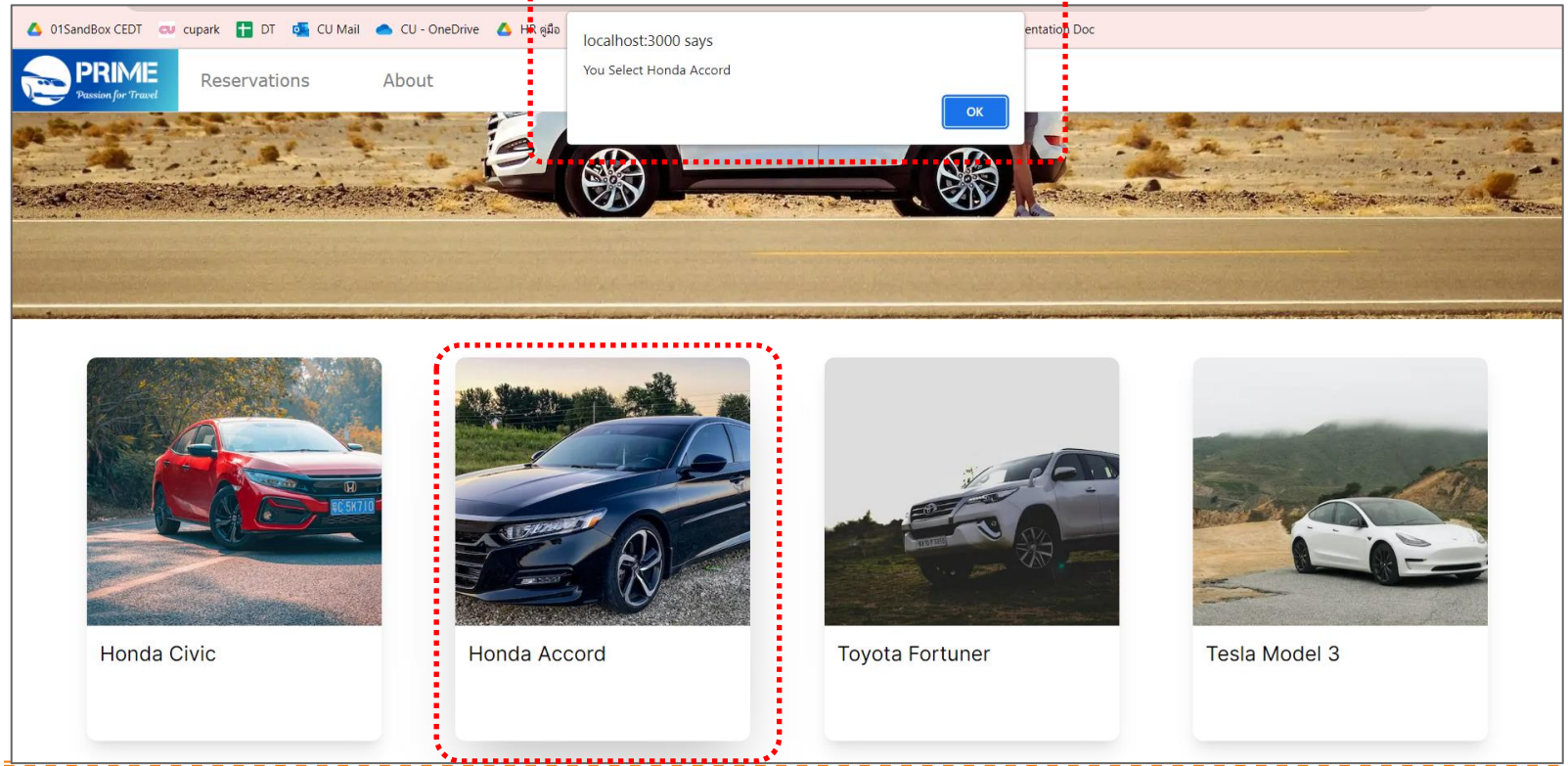
```
export default function ProductCard ( { carName, imgSrc } : { carName:string, imgSrc:string }) {

    function onCarSelected() {
        alert("You Select " + carName)
    }

    return (
        <div className='w-1/5 h-[300px] rounded-lg shadow-lg'
        onClick={()=>onCarSelected()}>
            <div className='w-full h-[70%] relative rounded-t-lg'>
            <Image src={imgSrc}
```

Interacting with Product Card

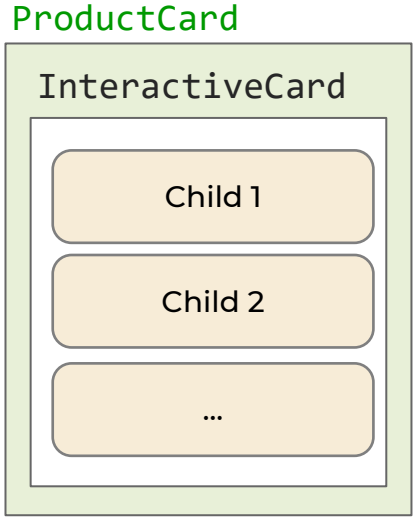
- เมื่อคลิก Card จะขึ้น alert



Interacting with Product Card

- แยกส่วนที่สามารถเป็น Server Component ได้ ออกจาก Card ที่เป็น Client Component
- เราจะแยก `ProductCard` component ที่มีอยู่เดิม ออกเป็น 2 ส่วน

  - `InteractiveCard` component เป็นตัว card ด้านนอก ที่มีเงา และรับ Interaction จาก user โดยเป็น Client Component
    - `InteractiveCard` รับ Server component เป็น props เข้ามาแสดงบน Card
  - `ProductCard` component ปรับกลับไปเป็น Server Component
    - บรรจุ `InteractiveCard` component เป็นกรอบนอกสุด
    - ภายใน `InteractiveCard` บรรจุ Image และ component อื่น ๆ



Interacting with Product Card



- สร้างไฟล์ `InteractiveCard.tsx` component จาก `ProductCard.tsx` ปรับเนื้อหาเป็น

```
'use client'
export default function InteractiveCard ({children} : {children : React.ReactNode}) {

    function onCardSelected() {
        alert("Card is Clicked")
    }

    return (
        <div className='w-1/5 h-[300px] rounded-lg shadow-lg'
        onClick={()=>onCardSelected()}>
            {children}
        </div>
    );
}
```

Interacting with Product Card

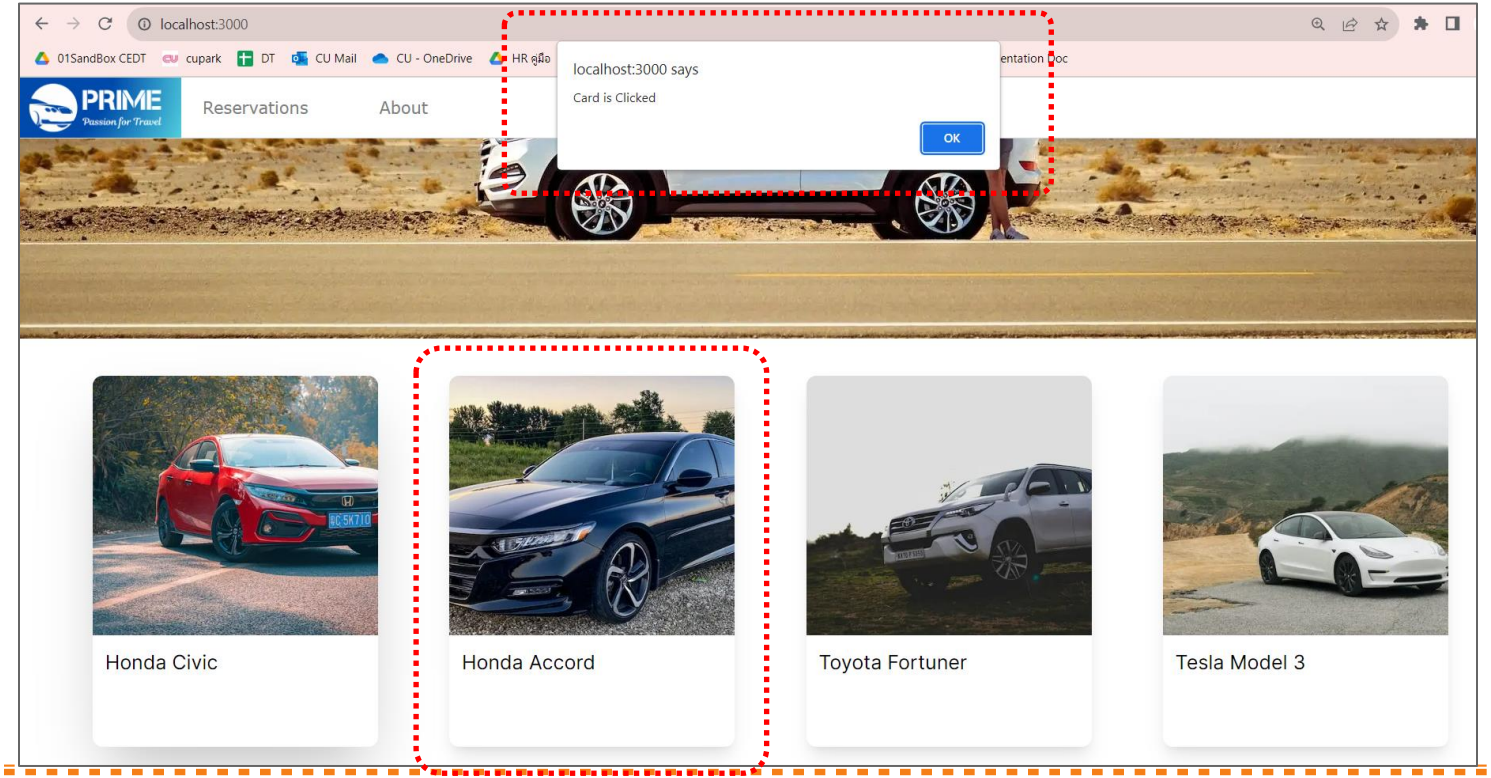


- ไฟล์ `ProductCard.tsx` ปรับเป็น Server Component และปรับเนื้อหาเป็น

```tsx
import Image from 'next/image';
import InteractiveCard from './InteractiveCard';
export default function ProductCard ( { carName, imgSrc } : { carName:string, imgSrc:string }) {
    return (
        <InteractiveCard>
            <div className='w-full h-[70%] relative rounded-t-lg'>
            <Image src={imgSrc}
                alt='Product Picture'
                fill={true}
                className='object-cover rounded-t-lg'/>
            </div>
            <div className='w-full h-[30%] p-[10px]'>{carName}</div>
        </InteractiveCard>
    );
}
```

Interacting with Product Card

- จะได้การแสดงผลตามเดิม และเมื่อคลิก Card จะขึ้น alert



Interacting with Product Card



- รับ Argument
- ไฟล์ `InteractiveCard.tsx` ให้รับชื่อเรียกสำหรับเนื้อหาที่จะแสดง มาจาก parent component
  - ใน function `onCardSelected` ให้ alert ชื่อเนื้อหามาด้วย

```
export default function InteractiveCard ({children, contentName} :
    {children : React.ReactNode, contentName:string}) {

    function onCardSelected() {
        alert("You Select " + contentName)
    }
```

  - ไฟล์ `ProductCard.tsx` ให้ส่งชื่อเรียกสำหรับเนื้อหาที่จะแสดง ไปให้ `InteractiveCard`

```
export default function ProductCard ( { carName, imgSrc } : { carName:strin
    return (
        <InteractiveCard contentName={carName}>
            <div className='w-full h-[70%] relative rounded-t-lg'>
            <Image src={imgSrc}
                alt='Product Picture'
```

Interacting with Product Card

ปรับรูปแบบการแสดงผลตาม event

- ไฟล์ `InteractiveCard.tsx` สร้างฟังก์ชั่น `onCardMouseAction` รับ Event Object เข้ามา

หากเป็น `mouseover` ให้

- เปลี่ยน CSS Tailwind class สำหรับ shadow ให้เป็น `shadow-2xl`

หากเป็นอย่างอื่นให้

- เปลี่ยน CSS Tailwind class สำหรับ shadow ให้เป็น `shadow-lg`

```
function onCardMouseAction(event:React.SyntheticEvent) {
    if(event.type=='mouseover')
    {
        event.currentTarget.classList.remove('shadow-lg')
        event.currentTarget.classList.add('shadow-2xl')
    }
    else {
        event.currentTarget.classList.remove('shadow-2xl')
        event.currentTarget.classList.add('shadow-lg')
    }
}

return (
    <div className='w-1/5 h-[300px] rounded-lg shadow-lg'
```

Tailwind CSS มี Utility Class ที่รองรับการใช้ modifier กับ Tailwind Class เพื่อ apply class ตามเงื่อนไข เช่น `hover:bg-sky-700` คือใช้ Class `bg-sky-700` เมื่อ Mouse Hover
อย่างไรก็ตาม Workshop นี้มุ่งนำเสนอ Mouse Event และการ add / remove Class จึงสาธิตการใช้งานคำสั่งข้างต้น

Interacting with Product Card

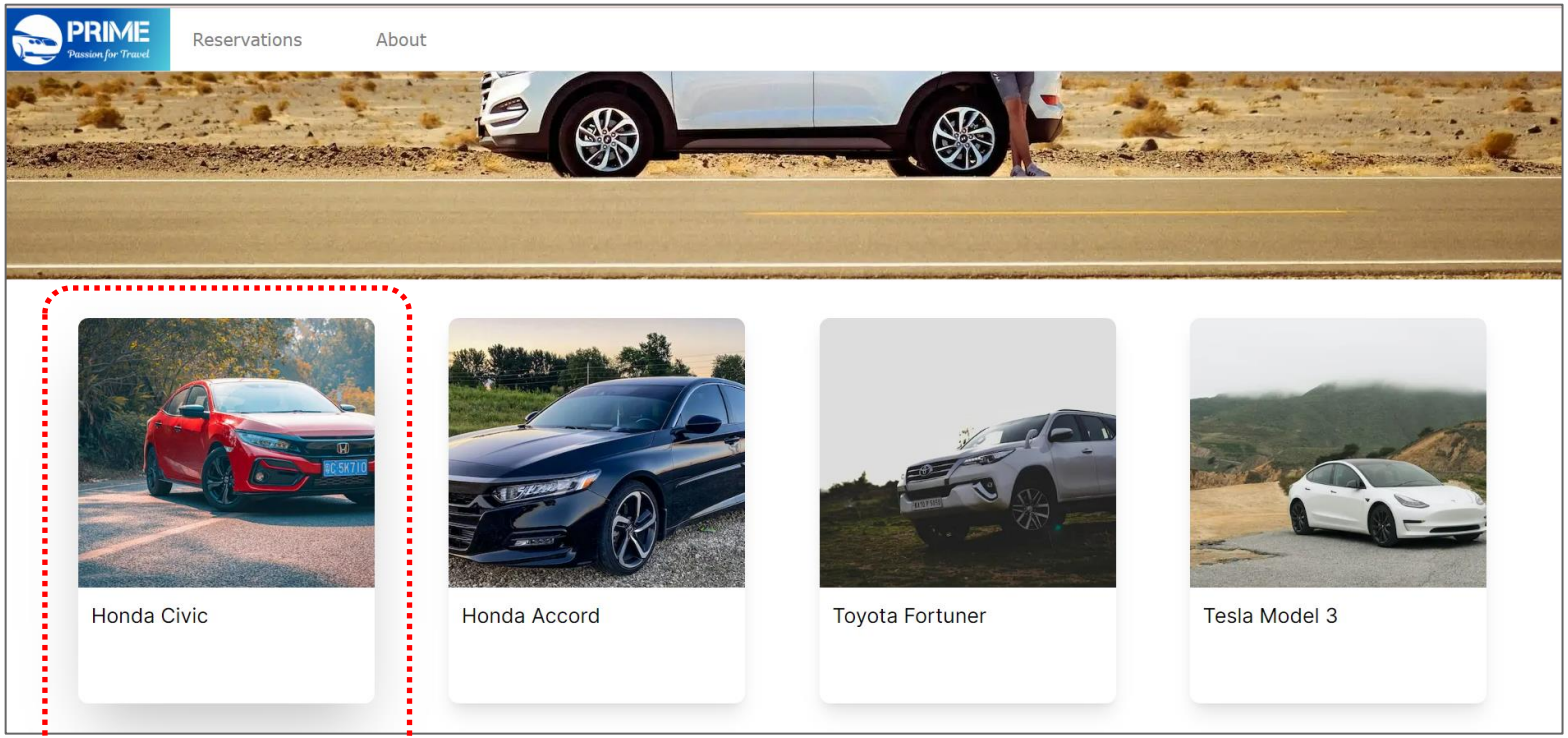


- ปรับรูปแบบการแสดงผลตาม event
- ไฟล์ `InteractiveCard.tsx` ให้ฟังก์ชั่น `onCardMouseAction` เป็น Event Handler สำหรับ `onMouseOver` และ `onMouseOut` ของ card โดยส่ง Event Object ไปให้

```
return (
    <div className='w-1/5 h-[300px] rounded-lg shadow-lg'
    onClick={()=>onCardSelected()}
    onMouseOver={(e)=>onCardMouseAction(e)}
    onMouseOut={(e)=>onCardMouseAction(e)}>
        {children}
    </div>
);
```

Interacting with Product Card

- จะได้การแสดงผลตามเดิม และเมื่อคลิกเอา mouse ไปทาบ เงาจะมีขนาดใหญ่ขึ้น



Form Components



- Tailwind CSS มาพร้อม UI components สำหรับสร้าง Application UI ต่าง ๆ เช่น Form layouts, tables, modal dialogs ฯลฯ
  - https://tailwindui.com/components#product-application-ui

  Form Components ใน Tailwind UI
  - https://tailwindui.com/components#product-application-ui-forms

- Developer สร้าง Form ผ่าน HTML element และ Style ด้วย Tailwind CSS
  - ตัวอย่างการสร้าง Text Input

```
<input type="text"
id="name"
placeholder="Your Name"
className="rounded ring-1 ring-inset ring-gray-400
text-md leading-6 indent-2 placeholder:text-gray-400"/>
```

Your Name

Form Components

- Tailwind CSS ที่นิยมนำมาปรับแต่ง Form Element
  - Background Color สีพื้นหลัง
    - https://tailwindcss.com/docs/background-color
  - Ring Width วงโดยรอบ พร้อม Box Shadow
    - ring-inset กำหนดให้วาด ring ไว้ด้านใน
    - https://tailwindcss.com/docs/ring-width
  - Ring Color สีของวงโดยรอบ
    - https://tailwindcss.com/docs/ring-color
  - Placeholder Color สีของข้อความที่เป็น Placeholder
    - https://v1.tailwindcss.com/docs/placeholder-color
  - Line Height ความสูงของบรรทัดของ Element
    - https://tailwindcss.com/docs/line-height
  - Text Indent ระยะเว้นด้านหน้าข้อความ
    - https://tailwindcss.com/docs/text-indent



Form Components



- ตัวอย่างการสร้าง Button

```
<button className="rounded-md bg-indigo-600 px-3 py-2
shadow-sm text-white">Hello</button>
```

Hello

- ตัวอย่างการสร้าง Checkbox

```
<input type="checkbox" id="dinner" checked
    className="h-4 w-4 rounded border-gray-300 focus:ring-indigo-600"/>
<label htmlFor="dinner" className="text-md text-gray-900"> Dinner</label>
```

Dinner

- ตัวอย่างการสร้าง Radio Button

```
<input id="gender" name="gender" type="radio" checked value="male"
 className="h-4 w-4 border-gray-300 text-indigo-600"/>
<label htmlFor="dinner" className="text-md text-gray-900"> Male</label>
<input id="gender" name="gender" type="radio" value="female"
 className="h-4 w-4 border-gray-300 text-indigo-600"/>
<label htmlFor="dinner" className="text-md text-gray-900"> Female</label>
```

Male Female

Form Components

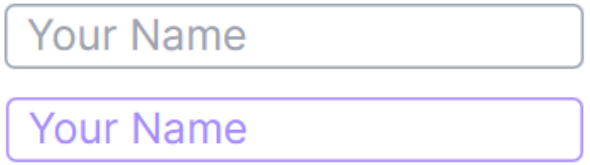
- การทำ Styling ให้ element ตาม Action
  - Tailwind CSS รองรับการกำหนด Styling สำหรับ element เมื่อมี action ต่าง ๆ บน element เช่น hover, focus, active, visited ฯลฯ
  - กำหนด class สำหรับ hover โดยกำหนด class เป็น `hover:class` เช่น
    - ปกติ ใช้ class `bg-indigo-600` เพื่อระบุสี background
    - เมื่อ `hover` ใช้ class `bg-violet-600` เพื่อระบุสี background




```
<button className="rounded-md bg-indigo-600 hover:bg-violet-600 px-3 py-2
shadow-sm text-white">Hello</button>
```

  - กำหนด class สำหรับ focus โดยกำหนด class เป็น `focus:class` เช่น

```
<input type="text"
id="name"
placeholder="Your Name"
className="rounded ring-1 ring-inset ring-gray-400
text-md leading-6 indent-2 placeholder:text-gray-400
focus:outline-none focus:ring-violet-400 focus:placeholder:text-violet-400"/><p></p>
```



Form Components

- UI Component อื่น ๆ จาก Library เช่น Material UI (https://mui.com/)
  - ติดตั้ง

```
npm install @mui/material @emotion/react @emotion/styled
```

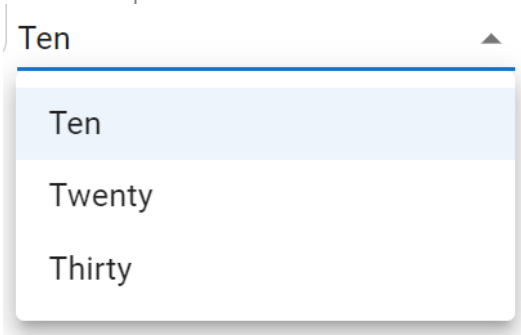
  - ตัวอย่างการใช้งาน **Select Drop Down List**
    - https://mui.com/material-ui/react-select/

```
import { Select, MenuItem } from "@mui/material"

export default function Form () {
    return (
        <form className="w-2/4 space-y-3">

            <Select variant="standard" name="amount"
            id="amount" className="w-2/4 h-[2em]">
                <MenuItem value={10}>Ten</MenuItem>
                <MenuItem value={20}>Twenty</MenuItem>
                <MenuItem value={30}>Thirty</MenuItem>
            </Select>

            <button className="block rounded-md bg-indigo-600 hover:bg-
```



Form Components

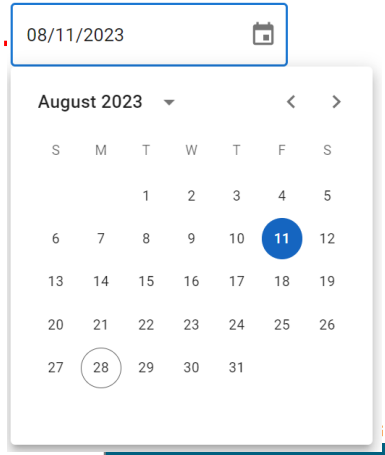
- ตัวอย่างการใช้งาน Date Picker
  - https://mui.com/x/react-date-pickers
  - หลังจากติดตั้ง MUI แล้ว ติดตั้ง DatePicker ผ่าน

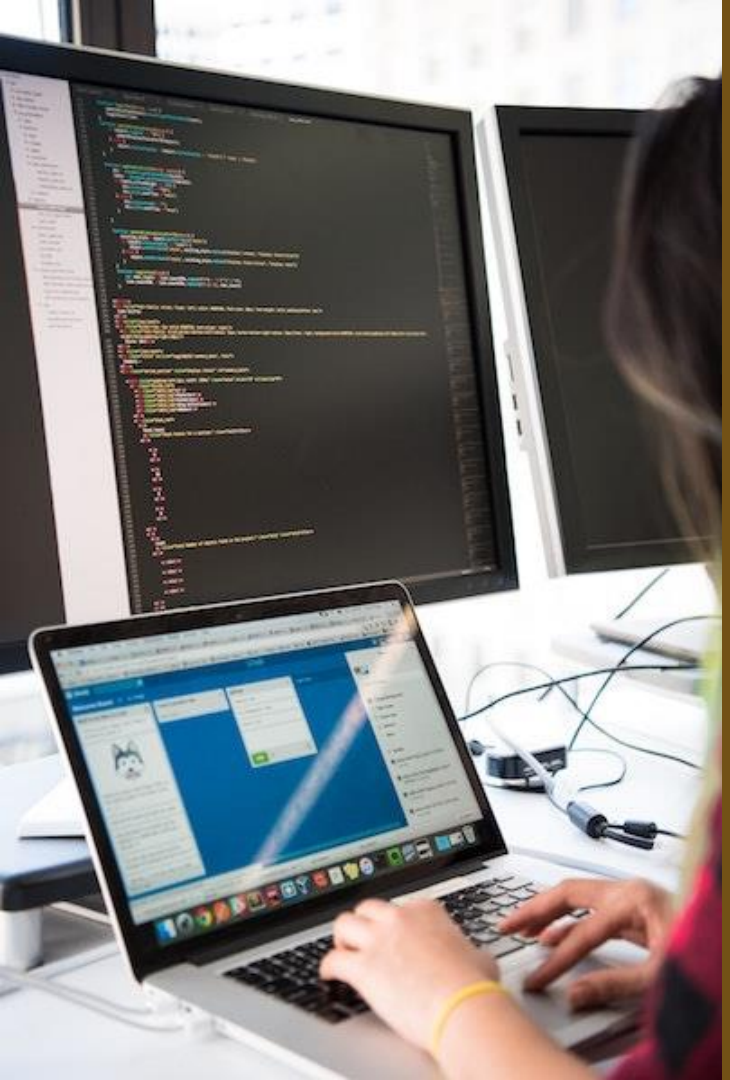
```
npm install dayjs
```

```
npm install @mui/x-date-pickers
```

```
"use client"
import { DatePicker } from "@mui/x-date-pickers"
import { LocalizationProvider } from "@mui/x-date-pickers/LocalizationProvider"
import { AdapterDayjs } from '@mui/x-date-pickers/AdapterDayjs'

export default function Form () {
    return (
        <form className="w-2/4 space-y-3">
        <LocalizationProvider dateAdapter={AdapterDayjs}>
            <DatePicker/>
        </LocalizationProvider>
        </form>
    )
}
```

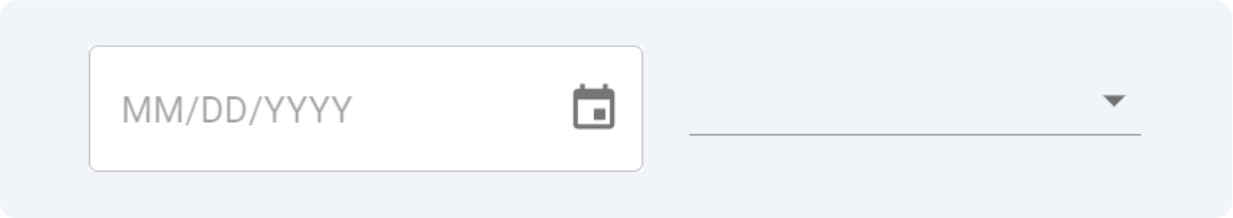




Workshop 5.2 : Form Components

Car Reservation Form

- สร้างหน้า components/LocationDateReserve.tsx สำหรับสร้าง Component ที่ประกอบด้วย Date Picker ให้เลือกวัน และ Drop Down List ให้เลือก location ของศูนย์เช่ารถ



- ติดตั้ง Material UI สำหรับใช้งาน Select และ Date Picker ผ่าน console

```
npm install @mui/material @emotion/react @emotion/styled
```

```
npm install @mui/x-date-pickers
```

```
npm install dayjs
```

Car Reservation Form

```
29  * {
30      box-sizing: border-box;
31      padding: 0;
32      margin: 0;
33  }
```

- ลบการระบุ padding และ margin เป็น 0 ใน global.css ออก
- สร้าง components/LocationDateReserve.tsx

```
1   "use client"
2   import { DatePicker } from "@mui/x-date-pickers"
3   import { LocalizationProvider } from "@mui/x-date-pickers/LocalizationProvider"
4   import { AdapterDayjs} from   "@mui/x-date-pickers/AdapterDayjs"
5   import { Select, MenuItem } from "@mui/material"
6
7   export default function LocationDateReserve() {
8       return (
9           <div className="bg-slate-100 rounded-lg gap-x-5 gap-y-2
10          w-fit px-10 py-5 flex flex-row justify-center">
11              <LocalizationProvider dateAdapter={AdapterDayjs}>
12                  <DatePicker className="bg-white"/>
13              </LocalizationProvider>
14              <Select variant="standard" name="location" id="location" className="h-[2em] w-[200px]">
15                  <MenuItem value="BKK">Bangkok</MenuItem>
16                  <MenuItem value="CNX">Chiang Mai</MenuItem>
17                  <MenuItem value="HKT">Phuket</MenuItem>
18              </Select>
19          </div>
20      )
21  }
```

Car Reservation Form

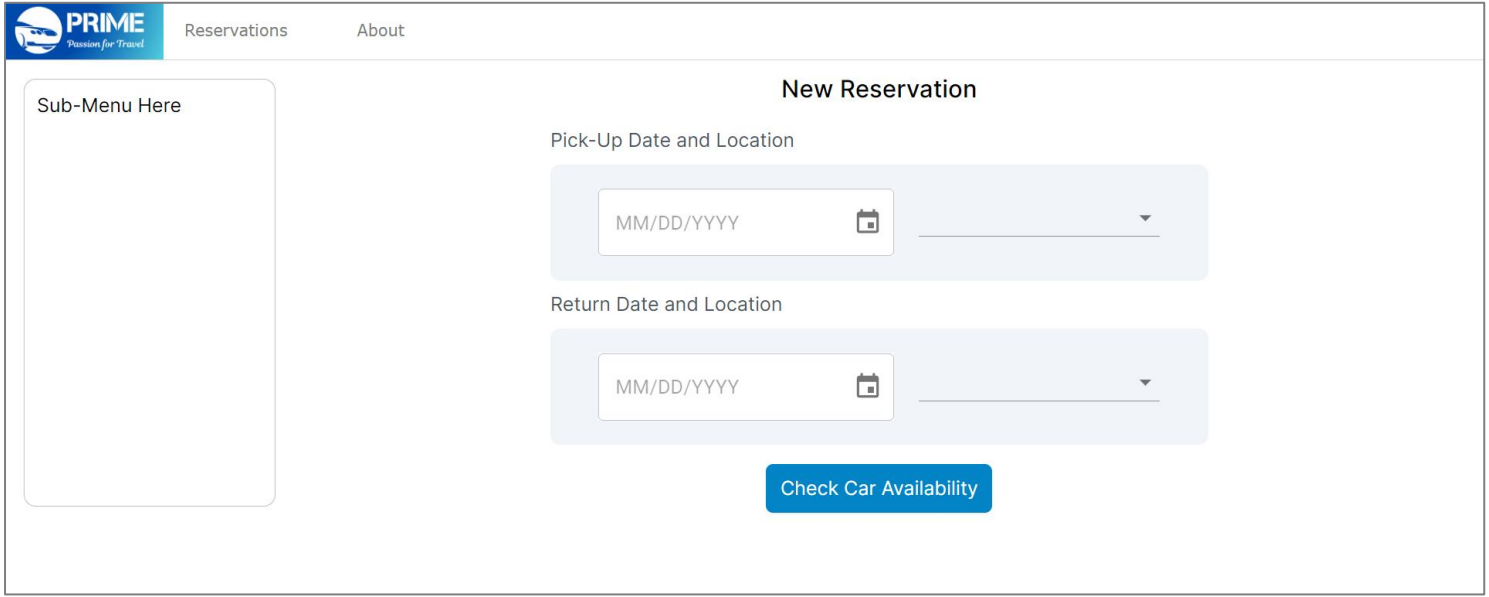


- สร้างหน้า Reservations/Page.tsx ใช้งาน LocationDateReserve component

```tsx
import LocationDateReserve from "@/components/LocationDateReserve";
export default function Reservations () {
    return (
        <main className="w-[100%] flex flex-col items-center space-y-4">
            <div className="text-xl font-medium">New Reservation</div>
            <div className="w-fit space-y-2">
                <div className="text-md text-left text-gray-600">
                    Pick-Up Date and Location</div>
                <LocationDateReserve/>
                <div className="text-md text-left text-gray-600">
                    Return Date and Location</div>
                <LocationDateReserve/>
            </div>
            <button className="block rounded-md bg-sky-600 hover:bg-indigo-600 px-3 py-2
            shadow-sm text-white">Check Car Availability</button>
        </main>
    );
}
```

Car Reservation Form

- ไปที่ http://localhost:3000/reservations



เนื่องจากตอนนี้ยังไม่มีการทำงานกับ Controlled State ทำให้การเลือกเปลี่ยนค่าในฟอร์มยังคงแสดง error เราจะเพิ่มการทำงานกับ State ในภายหลัง